

PROJETO DA DISCIPLINA × PIEX — PARTE 2

Financeiro e Frota Logística

Documentação Técnica do Núcleo Financeiro, Navegação, Arquitetura de Dados e Custos

Equipe: **Financeiro e Frota Logística**

Prazo de entrega: **05/10/2026**

Repositório: <https://github.com/benimaru0101-collab/frota-financeiro>

Sumário

1. Visão geral da entrega
2. Parte 1 — Implementação do Núcleo Financeiro
3. Parte 2 — Roteiro do vídeo demonstrativo
4. Parte 3 — Modelo de navegação e UX
5. Parte 4 — Arquitetura de dados, backend e custos
6. App nativo Android (APK)
7. Controle de versão
8. Testes realizados
9. Pendências e próximos passos

1. Visão geral da entrega

Esta entrega evolui o app “Financeiro e Frota Logística” (React Native + Expo + Supabase) com um núcleo financeiro mais rigoroso, controle de acesso por perfil e a documentação pedida no enunciado: modelo de navegação, arquitetura de dados, planejamento de custos e roteiro do vídeo demonstrativo. Além disso, o app passou a ser distribuído como aplicativo Android nativo (APK).

Requisito do enunciado	Status	Onde está
Status Pendente / Pago-Recebido, data e forma de pagamento	Concluído	db/migration-6 · Nova/Lista de Receitas e Despesas
Comprovante (foto ou galeria) no lançamento	Concluído	src/lib/upload.js · NovaReceita/NovaDespesa
Validação de valores e consistência	Concluído	Formulários + checks no banco
Dívidas e Financiamentos com parcelas e rollback	Concluído	db/migration-7 · DividasScreen · NovaDividaScreen
Motor de Saldo e Projeção de Caixa	Concluído	DataContext.js (resumo) · Dashboard
RBAC — Administrador × Motorista	Concluído	db/migration-8 · AuthContext · MainNavigator
Modelo de navegação e UX (Parte 3)	Documentado	Seção 4 deste documento
Arquitetura de dados, backend e custos (Parte 4)	Documentado	Seção 5 deste documento
Roteiro do vídeo (Parte 2)	Roteiro pronto	Seção 3 — gravação pela equipe
Repositório público + main protegida	Concluído	GitHub — regra “main” ativa

2. Parte 1 — Implementação do Núcleo Financeiro

2.1 Status, pagamento e comprovante

Toda receita e despesa passou a ter um status. No banco o valor é 'pendente' ou 'concluido'; na tela ele aparece como “Pendente”, “Recebido” (receitas) ou “Pago” (despesas). Os campos novos foram criados pela migration-6:

- status (obrigatório, com check que só aceita os dois valores);
- data_pagamento — preenchida ao marcar como pago/recebido e limpa se voltar para pendente;
- forma_pagamento — Pix, Boletto, Cartão, Dinheiro ou Transferência;
- comprovante_url — endereço da foto enviada ao Supabase Storage.

Os lançamentos que já existiam foram tratados como concluídos, para não zerar o saldo histórico. No formulário, a forma de pagamento e o comprovante só aparecem quando o lançamento já está pago; na lista, um selo colorido mostra o status e o toque-e-segure oferece “Marcar como Pago/Recebido”, “Marcar como Pendente” ou “Excluir”.

2.2 Validações

- O valor precisa ser maior que zero; caso contrário o app mostra um aviso e não salva.
- O app não permite salvar enquanto o comprovante ainda está sendo enviado.
- No banco, o check de status e as restrições da função de dívidas impedem dados inconsistentes mesmo que alguém chame a API diretamente.

2.3 Dívidas e Financiamentos

A migration-7 criou a tabela `dividas_financiamentos` (descrição, credor, valor total, número de parcelas de 1 a 360, juros informativos, data de início, categoria e veículo opcionais) e a coluna `divida_id` nas despesas, que liga cada parcela à sua dívida.

O cadastro chama a função `criar_divida_com_parcelas` no Postgres. Em uma única chamada ela cria a dívida e gera cada parcela como uma despesa “Pendente”, mês a mês. A última parcela absorve a diferença de arredondamento (ex.: 1.000 em $3x = 333,33 + 333,33 + 333,34$).

Rollback: funções PL/pgSQL rodam dentro de uma transação. Se qualquer parcela falhar no meio da geração, o Postgres desfaz tudo — a dívida e as parcelas já inseridas — e nada fica gravado pela metade. Isso foi testado com uma falha simulada na parcela 3 de 5 (seção 8).

No app, a tela “Dívidas e Financiamentos” lista cada dívida com uma barra de progresso das parcelas pagas, e a tela “Nova Dívida/Financiamento” faz o cadastro.

2.4 Motor de Saldo e Projeção de Caixa

- **Saldo Atual:** considera apenas lançamentos concluídos (Pago/Recebido) — é o card principal do Dashboard. Pendências não mexem no saldo real.
- **Projeção de Caixa:** agrupa os lançamentos pendentes pelo mês de vencimento e mostra, para cada mês, quanto há a receber, a pagar e o saldo projetado. As parcelas geradas pelas dívidas entram automaticamente aqui.

2.5 Controle de acesso — Administrador × Motorista

A migration-8 adicionou o campo `role` em perfis ('administrador' ou 'motorista'). O perfil é criado automaticamente no primeiro login, como administrador por padrão; usuários que já existiam também receberam perfil de administrador, para não perderem acesso.

- **No app:** o Motorista não vê a aba Financeiro, nem o saldo, nem os atalhos financeiros do Dashboard; a tela de Perfil mostra o papel do usuário.
- **No banco:** a função `is_admin()` é usada nas policies de RLS das cinco tabelas financeiras (receitas, despesas, contas, categorias e dívidas). Assim, mesmo uma chamada direta à API não entrega dados financeiros a quem não é administrador.
- **Limitação conhecida:** a tabela motoristas ainda não está ligada a usuários do sistema, então o Motorista enxerga a Frota inteira (nunca o Financeiro). Restringir “só as próprias viagens” fica para uma próxima entrega.

3. Parte 2 — Roteiro do vídeo demonstrativo

Sugestão de roteiro (cerca de 5 minutos), gravado no emulador Android ou em um celular com o APK instalado. Antes de gravar, confira se o projeto do Supabase está ativo (o plano gratuito pausa após um período sem uso).

1. **0:00** — Apresentar o app e a equipe; mostrar o ícone instalado e abrir o app (tela de boas-vindas).
2. **0:30** — Fazer login como Administrador e mostrar o Dashboard: Saldo Atual, receitas/despesas e Projeção de Caixa.
3. **1:00** — Cadastrar uma despesa Pendente (ex.: manutenção com vencimento no mês seguinte) e mostrar que ela entra na Projeção, mas não altera o Saldo Atual.
4. **1:45** — Cadastrar uma receita já Recebida, escolhendo a forma de pagamento e anexando um comprovante pela câmera ou galeria; mostrar o Saldo Atual subindo.
5. **2:30** — Na lista de Despesas, tocar e segurar na despesa pendente e marcar como Paga; voltar ao Dashboard e mostrar o saldo atualizado.

6. **3:00** — Abrir Dívidas e Financiamentos e cadastrar, por exemplo, R\$ 1.000 em 3x; mostrar as três parcelas geradas em Despesas e na Projeção de Caixa.
7. **3:45** — Mostrar o controle de acesso: com um usuário de papel Motorista, a aba Financeiro não aparece.
8. **4:30** — Encerrar mostrando o repositório público e a regra de proteção da main (Pull Request + 1 aprovação).

4. Parte 3 — Modelo de navegação e UX

O app usa React Navigation com Bottom Tabs (barra inferior) e uma pilha de telas (Stack) dentro de cada aba. As abas são Início, Financeiro, Frota e Mais. Em cada aba o fluxo é previsível: lista → detalhe → formulário, com os formulários abertos como modal.

Aba	Principais telas	Administrador	Motorista
Início	Dashboard: saldo, projeção, avisos de vencimento, atalhos	Completo	Só frota e avisos
Financeiro	Receitas, Despesas, Fluxo de Caixa, Contas, Categorias, Relatórios, Dívidas	Sim	Oculto
Frota	Veículos, Motoristas, Viagens, Abastecimentos, Manutenções, Documentos	Sim	Sim
Mais	Perfil (com o papel), Configurações, Editar perfil, Segurança	Sim	Sim

Justificativa

- **Acesso rápido:** a barra inferior deixa as quatro áreas sempre a um toque, o que é adequado para uso rápido no celular, inclusive por motoristas em trânsito.
- **Contexto preservado:** cada aba guarda a sua própria pilha, então o usuário não perde o lugar ao alternar entre Financeiro e Frota.
- **Um app, dois perfis:** em vez de dois apps ou dois fluxos, existe um único fluxo filtrado pelo papel. O Motorista vê menos opções — menos poluição visual e menos risco de mexer em dados financeiros —, e a manutenção fica mais simples.
- **Consistência visual:** tema escuro com um único acento amarelo nas ações principais, alto contraste e selos de status coloridos (verde = concluído, amarelo = pendente).

5. Parte 4 — Arquitetura de dados, backend e custos

5.1 Arquitetura

O app segue o modelo Backend-as-a-Service: o aplicativo React Native conversa diretamente com o Supabase pelo SDK oficial, sem um servidor de API próprio.

- **Banco:** PostgreSQL gerenciado com as tabelas da frota e do financeiro.
- **Autenticação:** Supabase Auth com e-mail/senha e login com Google (OAuth2 + PKCE); sessão persistida no aparelho.
- **Arquivos:** Supabase Storage (bucket “documentos”) para arquivos da frota e comprovantes.
- **Regras e segurança:** Row Level Security em todas as tabelas; regras financeiras restritas a administradores via `is_admin()`; geração de parcelas como função (RPC) transacional.

Não foi criado um backend próprio porque o Supabase já expõe uma API segura sobre o banco; um servidor extra só duplicaria autenticação e regras sem trazer benefício para o escopo do projeto.

5.2 Modelo de dados

Tabela	Função
profiles	Perfil do usuário (1:1 com auth.users) e papel (role)
veiculos, motoristas, viagens	Cadastro da frota e das viagens
abastecimentos, manutencoes, documentos	Custos e documentos por veículo (com vencimento)
categorias_financeiras, contas	Estrutura do financeiro
receitas, despesas	Lançamentos com status, pagamento e comprovante
dividas_financiamentos	Dívidas; as parcelas são despesas ligadas por divida_id

As mudanças de banco ficam versionadas em arquivos de migração dentro de db/ (schema.sql e migrations 2 a 8), aplicados no SQL Editor do Supabase.

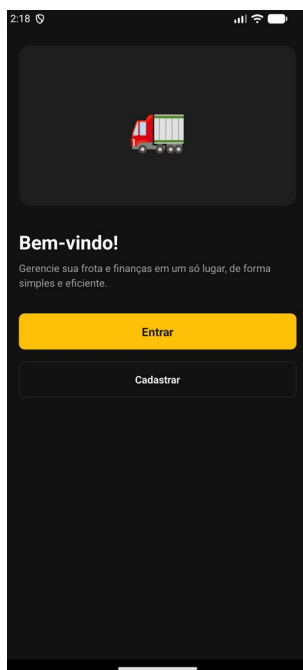
5.3 Hospedagem e planejamento de custos

Serviço	Plano	Custo mensal	Observação
Supabase	Free (Postgres 500 MB, Auth, Storage 1 GB)	R\$ 0,00	Suficiente para o escopo acadêmico
Expo / EAS	Free	R\$ 0,00	APK gerado localmente; EAS Build gratuito como alternativa
Google Cloud (OAuth)	Client ID gratuito	R\$ 0,00	Usado só para o login com Google
GitHub	Free (repositório público)	R\$ 0,00	Regras de proteção valem em repositório público
Total atual		R\$ 0,00	

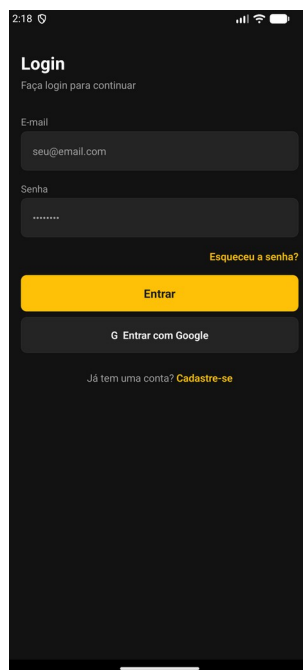
Riscos e evolução: o plano gratuito do Supabase pausa o projeto após um período sem uso (aconteceu durante esta entrega e foi preciso reativá-lo). Para uso real, o próximo passo seria o plano Pro do Supabase (a partir de US\$ 25/mês), que não pausa e amplia banco e armazenamento; publicar na Play Store teria a taxa única da conta de desenvolvedor Google.

6. App nativo Android (APK)

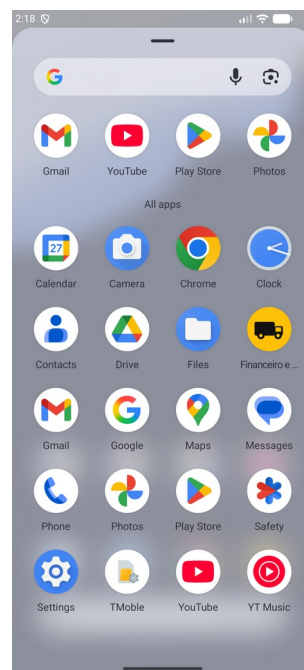
O app foi compilado como APK de release (Expo prebuild + Gradle) e instalado no emulador Android (API 36), onde abriu normalmente até a tela de login.



Boas-vindas



Login (e-mail ou Google)



Ícone instalado

Como gerar e instalar

1. Criar o arquivo .env (a partir de .env.example) com SUPABASE_URL e SUPABASE_ANON_KEY — ele não vai para o Git; o app.config.js lê essas variáveis no build.
2. Build local: `npx expo prebuild --platform android --clean` e depois, na pasta android, `gradlew assembleRelease`. O APK sai em `android/app/build/outputs/apk/release/`.
3. Alternativa na nuvem: `npx eas-cli build -p android --profile preview` (conta Expo gratuita).
4. Instalar no celular (permitindo “fontes desconhecidas”) ou no emulador com `adb install`.

7. Controle de versão

- **Repositório:** repositório público no GitHub, com todo o histórico de commits.
- **Proteção da main:** regra ativa na branch principal — Pull Request obrigatório com 1 aprovação, sem push direto, sem force push e sem apagar a branch.
- **Fluxo:** as mudanças do build do APK estão no Pull Request #2, aguardando a aprovação de um colega.
- **Segredos:** chaves reais ficam em .env (fora do Git); o histórico foi verificado e não contém credenciais.

8. Testes realizados

Teste	Resultado
Schema + migrations 2 a 8 em um PostgreSQL 16 local	Aplicadas sem erro; migrations 6, 7 e 8 podem rodar mais de uma vez
Dívida de 1.000 em 3x	$333,33 + 333,33 + 333,34 = 1.000,00$
Falha simulada na parcela 3 de 5	Rollback: nenhuma parcela nem a dívida gravadas
Valor total negativo	Recusado pela função, nada gravado
Usuário Motorista acessando receitas	0 registros visíveis; inserção recusada pela RLS
Sintaxe e importações de todo o src/	Todos os arquivos válidos, todas as importações resolvidas

Teste	Resultado
Migrations no Supabase de produção	Aplicadas e conferidas (colunas, função, 20 policies)
Build do APK e execução no emulador	BUILD SUCCESSFUL; app abre até o login

9. Pendências e próximos passos

1. Testar o fluxo completo com um usuário logado (emulador ou celular).
2. Gravar o vídeo seguindo o roteiro da seção 3.
3. Um colega aprovar o Pull Request #2 no GitHub.
4. Próxima evolução: ligar motoristas a usuários para restringir as viagens de cada motorista.